Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №2

по дисциплине «Сервис-ориентированная архитектура»

Реализация веб-сервисов на JAX-RS и Spring MVC REST и клиентского приложения

Вариант: 99

Выполнили:
Молодиченко Семён Андреевич, группа P3413
Шаматульский Роман Константинович, группа P3412

Преподаватель:
Коновалов Арсений Антонович

Санкт-Петербург
2026

Содержание

1. Текст задания 3
1.1. Результаты работы 3
2. Предметная область и спецификация 4
3. Архитектура и развёртывание 4
4. Первый веб-сервис (JAX-RS, WildFly) 5
4.1. Структура 5
4.2. Операции 6
4.3. Сортировка, фильтрация и постраничный вывод 6
4.4. Проверка данных и ошибки 7
4.5. Хранение данных и CORS 7
5. Второй веб-сервис (Spring MVC REST, Tomcat) 8
5.1. Операции 8
5.2. Вызов первого сервиса 8
5.3. Развёртывание в Tomcat 8
6. Клиентское приложение 9
6.1. Технологии и сборка 9
6.2. Возможности 9
6.3. Человекочитаемое представление 9
6.4. Сообщения об ошибках 9
7. HTTPS и самоподписанные сертификаты 10
7.1. Сертификаты 10
7.2. WildFly 10
7.3. Tomcat 10
7.4. Доверие между участниками 11
8. Конфигурация серверов и развёртывание на helios 11
8.1. WildFly 41.0.1.Final 11
8.2. Tomcat 11.0.27 11
8.3. Сборка и запуск 12
9. Проверка 12
10. Выводы 13
11. Источники 13
Приложение А. Выполнение пунктов задания 14
Приложение Б. Структура репозитория 14

1. Текст задания

На основе разработанной в рамках лабораторной работы №1 спецификации реализовать два веб-сервиса и использующее их API клиентское приложение.

Требования к реализации и развёртыванию сервисов:

1. Первый («вызываемый») веб-сервис должен быть реализован на фреймворке JAX-RS и развёрнут в окружении под управлением сервера приложений WildFly.
2. Второй веб-сервис должен быть реализован на фреймворке Spring MVC REST, развёрнут в окружении под управлением сервера приложений Tomcat и вызывать REST API первого сервиса.
3. Для обоих сервисов необходимо реализовать все функции, задокументированные в API, в строгом соответствии со спецификацией.
4. Доступ к обоим сервисам должен быть реализован по протоколу https с самоподписанным сертификатом сервера. Доступ к сервисам посредством http без шифрования должен быть запрещён.

Требования к клиентскому приложению:

1. Клиентское приложение может быть написано на любом веб-фреймворке, который можно запустить на сервере helios.
2. Приложение должно обеспечить полный набор возможностей, предоставляемых API обоих сервисов, включая сортировку, фильтрацию и постраничный вывод элементов коллекции.
3. Приложение должно преобразовывать передаваемые сервисами данные в человекочитаемый вид: параграф текста, таблицу и т. д.
4. Клиентское приложение должно информировать пользователя об ошибках, возникающих на стороне сервисов, в частности о том, что сервису были отправлены невалидные данные.

Оба веб-сервиса и клиентское приложение должны быть развёрнуты на сервере helios.

1.1. Результаты работы

Таблица 1 — Где находятся результаты

Объект	Адрес / файл
Исходный код (Gradle, три модуля)	https://github.com/xxxxxxhs/soa_lab2
Первый сервис (JAX-RS)	vehicles-service → vehicles.war; WildFly: standalone/deployments/vehicles.war
Второй сервис (Spring MVC REST)	shop-service → ROOT.war; Tomcat: webapps/ROOT.war
Клиентское приложение	client → client.war; Tomcat: webapps/client.war (контекст /client)
Конфигурация WildFly	bin/standalone.conf, standalone/configuration/standalone.xml, modules/org/postgresql/main/module.xml
Конфигурация Tomcat	bin/setenv.sh, conf/server.xml
Ключи и сертификаты	vehicles.p12, vehicles.crt (WildFly); shop.p12, shop.crt (Tomcat)

2. Предметная область и спецификация

Спецификация лабораторной работы №1 описывает два сервиса. Первый управляет коллекцией транспортных средств Vehicle. Второй — автомастерская (shop) — изменяет объекты коллекции, вызывая API первого сервиса. Оба сервиса принимают и отдают данные в формате application/xml. Ошибка в любом сервисе возвращается как элемент <error> с полями code, message и time.

Таблица 2 — Поля класса Vehicle и ограничения, реализованные в сервисе

Поле	Тип	Ограничения
id	Integer	Больше 0, уникальное, генерируется базой данных
name	String	Обязательное, строка не может быть пустой
coordinates.x	Integer	Если не передано — 0
coordinates.y	Long	Обязательное, больше −943
creationDate	OffsetDateTime	Генерируется сервисом при создании; в XML — ISO 8601 со смещением
enginePower	Long	Обязательное, больше 0
capacity	Double	Обязательное, больше 0, конечное число
distanceTravelled	Float	Обязательное, не меньше 0, конечное число
type	VehicleType	Обязательное: PLANE, HELICOPTER, BOAT, SHIP, BICYCLE
numberOfWheels	Integer	Не меньше 0; если не передано — 0

Значения VehicleType упорядочены по объявлению: PLANE < HELICOPTER < BOAT < SHIP < BICYCLE. Этот порядок используется при сортировке по полю type и в операции «тип меньше заданного».

3. Архитектура и развёртывание

Таблица 3 — Компоненты системы

Компонент	Технологии	Сервер	Адрес (локально)
vehicles-service	Jakarta EE 11: JAX-RS, CDI, JPA, Bean Validation, JAXB	WildFly 41.0.1.Final	https://localhost:8443/api/v1/vehicles
shop-service	Spring Boot 4.1 (Spring MVC REST, RestClient), JAXB	Tomcat 11.0.27	https://localhost:9443/api/v1/shop
client	Vue 3.5, Vite 8	Tomcat 11.0.27	https://localhost:9443/client/
База данных	PostgreSQL, JDBC-драйвер 42.7.14	—	jdbc:postgresql://localhost:5432/studs

Проект собирается Gradle и состоит из трёх модулей: vehicles-service, shop-service и client. Java-модули компилируются под Java 17, клиент собирается Vite внутри Gradle. Схема взаимодействия компонентов показана на рисунке 1.

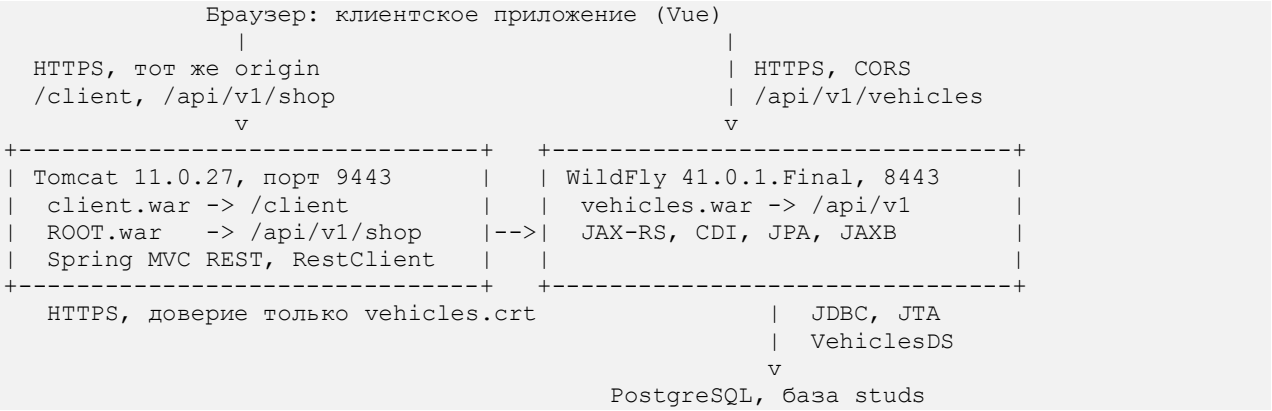


Рисунок 1 — Схема взаимодействия компонентов

Клиент загружается с Tomcat и обращается к автомастерской по относительному адресу /api/v1/shop, то есть на тот же origin. К сервису коллекции клиент обращается напрямую на WildFly, поэтому первый сервис отвечает с заголовками CORS (раздел 4.5).

Операция автомастерской проходит так: клиент отправляет PATCH на shop-service; тот вызывает API первого сервиса и отвечает 303 See Other с заголовком Location, указывающим на изменённый объект в сервисе коллекции. Браузер по правилам 303 повторяет запрос методом GET по этому адресу и получает актуальное состояние объекта, которое клиент показывает пользователю.

4. Первый веб-сервис (JAX-RS, WildFly)

4.1. Структура

Приложение JAX-RS объявлено классом VehiclesApplication с @ApplicationPath("/api/v1"); WEB-INF/jboss-web.xml задаёт контекст «/», поэтому ресурсы доступны по адресу /api/v1/vehicles.

Таблица 4 — Пакеты сервиса коллекции

Пакет	Содержимое
api	VehicleResource — ресурс /vehicles; PathParams — разбор параметров пути; CorsFilter
service	VehicleService — бизнес-логика и запросы к БД (CDI, @Transactional)
model	Сущность JPA Vehicle, встраиваемый Coordinates, перечисление VehicleType
dto	VehicleInput (POST, PUT), VehiclePatch (PATCH), VehiclePage, VehicleList, CountResult, ErrorResponse
query	Разбор параметров sort, filter, page, size; поля и операторы фильтрации
validation	RequestValidator (Bean Validation → 422), ограничение @Finite
xml	StrictXmlBodyReader — строгий разбор XML; адаптеры даты и VehicleType
error	ApiException и мапперы исключений в XML-ответ <error>

4.2. Операции

Таблица 5 — Операции сервиса коллекции (префикс /api/v1)

Метод	URL	Назначение	Ответы
GET	/vehicles	Массив элементов: сортировка, фильтрация, страницы	200 vehiclePage; 204 — пусто или страница вне выборки; 400; 422
POST	/vehicles	Добавить элемент	201 + Location; 400; 415; 422
GET	/vehicles/{id}	Получить по ID	200; 400; 404
PUT	/vehicles/{id}	Заменить элемент целиком	200; 400; 404; 422
PATCH	/vehicles/{id}	Изменить только переданные поля	200; 400; 404; 422
DELETE	/vehicles/{id}	Удалить элемент	204; 400; 404
GET	/vehicles/max-engine-power	Элемент с максимальной мощностью двигателя	200; 204 — коллекция пуста
GET	/vehicles/distance-travelled/greater-than/{distanceTravelled}/count	Количество элементов с пробегом больше заданного	200 countResult; 400
GET	/vehicles/type/less-than/{type}	Элементы с типом меньше заданного	200 vehicles; 204 — нет подходящих; 400

Операция задаётся методом HTTP, а все параметры передаются в URL. Если у нескольких элементов одинаковая максимальная мощность, max-engine-power возвращает элемент с меньшим ID. На любой ресурс сервис также может ответить 405 (метод не поддерживается, с заголовком Allow), 406 (клиент не принимает application/xml) и 415 (тело не в application/xml).

4.3. Сортировка, фильтрация и постраничный вывод

Таблица 6 — Параметры запроса GET /vehicles

Параметр	Формат	Правила
sort	поле[,поле…]; «-» перед полем — по убыванию	Любая комбинация полей, порядок важен. Неизвестное или повторённое поле — 400
filter	поле[оператор]=значение; параметр повторяется	Условия объединяются через И. Операторы eq, ne, gt, gte, lt, lte; для type — только eq и ne (иначе 422). Значение не того типа — 422, нарушение формата — 400
page	целое число	По умолчанию 1, не меньше 1
size	целое число	По умолчанию 20, от 1 до 100

Поля для sort и filter: id, name, coordinates.x, coordinates.y, creationDate, enginePower, capacity, distanceTravelled, type, numberOfWheels. Для параметров page и size нецелое значение даёт 400, а значение вне допустимого диапазона — 422. Пример запроса (в URL квадратные скобки и знак «=» внутри filter кодируются):

```
GET /api/v1/vehicles?sort=-enginePower,name&filter=type[eq]=BOAT
                    &filter=capacity[gte]=2.5&page=1&size=10
```

Запрос строится через Criteria API. Сначала считается общее число подходящих элементов; если смещение страницы не меньше этого числа, возвращается 204. В конец сортировки всегда добавляется id по возрастанию, чтобы порядок на страницах был однозначным. Тип хранится строкой, поэтому сортировка по type идёт через выражение CASE по порядку объявления. Для столбца name задано сравнение collate "C", и порядок

строк не зависит от локали базы данных. Структура ответа (элемент vehicle взят из модульного теста XmlMappingTest):

```
<vehiclePage>
  <vehicles>
    <vehicle>
      <id>1</id><name>Aurora</name>
      <coordinates><x>10</x><y>20</y></coordinates>
      <creationDate>2026-10-08T15:36:00+03:00</creationDate>
      <enginePower>150</enginePower><capacity>4.5</capacity>
      <distanceTravelled>1500.5</distanceTravelled><type>BOAT</type>
      <numberOfWheels>4</numberOfWheels>
    </vehicle>
  </vehicles>
  <page>1</page><size>10</size>
  <totalElements>1</totalElements><totalPages>1</totalPages>
</vehiclePage>
```

4.4. Проверка данных и ошибки

Таблица 7 — Обработка некорректных запросов

Ситуация	Код	Где обрабатывается
Тело не является корректным XML, неизвестный элемент или значение не того типа	400	StrictXmlBodyReader: JAXB с обработчиком, отклоняющим любое событие разбора
Нарушены ограничения полей (таблица 2)	422	RequestValidator: все нарушения в одном сообщении вида «поле: причина; …»
NaN или бесконечность в capacity, distanceTravelled	422	Ограничение @Finite
PATCH без единого поля	422	VehicleService.patch
ID в пути не целое число	400	PathParams
Элемент не найден (в том числе ID ≤ 0)	404	VehicleService.get
Неизвестный URL, неподдерживаемый метод или Content-Type	404, 405, 406, 415	WebApplicationExceptionMapper
Непредвиденная ошибка	500	UnhandledExceptionMapper, запись в журнал сервера

Все ошибки возвращаются в одном формате. В StrictXmlBodyReader отключены DTD и внешние сущности, что закрывает атаку XXE. Формат ответа об ошибке:

```
<error>
  <code>422</code>
  <message>enginePower: значение должно быть больше 0</message>
  <time>…</time>
</error>
```

4.5. Хранение данных и CORS

Единица персистентности vehicles использует JTA-источник данных java:jboss/datasources/VehiclesDS. Он описан в standalone.xml и подключён к PostgreSQL через модуль WildFly org.postgresql. Таблица vehicles создаётся и обновляется Hibernate (hibernate.hbm2ddl.auto = update), ID генерируется столбцом IDENTITY. VehicleService помечен @ApplicationScoped и @Transactional, поэтому каждая операция выполняется в своей транзакции. Дата создания проставляется сервисом с точностью до секунды.

CorsFilter (@Provider, @PreMatching) отвечает на предварительные запросы OPTIONS и добавляет к ответам заголовки Access-Control-Allow-Origin: *, список методов GET, POST, PUT, PATCH, DELETE, OPTIONS и заголовков Content-Type, Accept. Заголовок Location

открыт через Access-Control-Expose-Headers: после POST клиент читает адрес созданного ресурса.

5. Второй веб-сервис (Spring MVC REST, Tomcat)

5.1. Операции

Таблица 8 — Операции автомастерской

Метод	URL	Назначение	Ответы
PATCH	/api/v1/shop/fix-distance/{vehicle-id}	Скрутить пробег: distanceTravelled = 0	303 + Location; 400; 404; 422; 502; 503
PATCH	/api/v1/shop/add-wheels/{vehicle-id}/{number-of-wheels}	Увеличить numberOfWheels на заданное число	303 + Location; 400; 404; 422; 502; 503

Запрос проходит цепочку ShopController → ShopService → VehiclesClient. fix-distance отправляет первому сервису PATCH с единственным полем distanceTravelled, равным 0. add-wheels сначала получает элемент запросом GET, складывает число колёс через Math.addExact (переполнение int — 422) и отправляет PATCH с новым numberOfWheels. Параметр number-of-wheels должен быть целым (иначе 400) и больше 0 (иначе 422). При успехе сервис отвечает 303 See Other с Location на изменённый элемент в сервисе коллекции.

5.2. Вызов первого сервиса

VehiclesClient построен на RestClient поверх java.net.http.HttpClient. SSL-контекст берётся из SSL-бандла Spring Boot «vehicles», в доверенных у которого только сертификат первого сервиса:

```
vehicles.base-url=${VEHICLES_BASE_URL:https://localhost:8443/api/v1}
spring.ssl.bundle.pem.vehicles.truststore.certificate=file:${VEHICLES_CERT}
spring.web.resources.add-mappings=false
```

Адрес первого сервиса и путь к сертификату задаются переменными окружения, поэтому один и тот же WAR работает и локально, и на helios. Таймаут соединения — 5 секунд. Ответы первого сервиса преобразуются так:

Таблица 9 — Трансляция ответов первого сервиса

Ответ первого сервиса	Ответ автомастерской
4xx (400, 404, 422 …)	Тот же код, сообщение из <error> первого сервиса
5xx	502 «Сервис коллекции вернул ошибку»
Нет соединения, таймаут, ошибка TLS	503 «Сервис коллекции недоступен»

Ошибки самого Spring MVC (404, 405, 406, 415) ErrorHandler (@RestControllerAdvice) возвращает в том же XML-формате <error>, что и первый сервис. Остальные исключения дают 500 и пишутся в журнал.

5.3. Развёртывание в Tomcat

shop-service собирается как обычный WAR: ShopApplication наследует SpringBootServletInitializer, встроенный Tomcat исключён из зависимостей, задачи bootWar и jar отключены. Архив называется ROOT.war, поэтому Tomcat разворачивает его в корневом

контексте, а контроллер отвечает по /api/v1/shop. Параметр spring.web.resources.add-mappings=false отключает раздачу статических ресурсов: запрос на неизвестный адрес получает 404 в формате XML.

6. Клиентское приложение

6.1. Технологии и сборка

Клиент написан на Vue 3.5 и собирается Vite 8; других библиотек нет. Запросы выполняются через fetch, XML разбирается DOMParser. Gradle-плагин node-gradle скачивает Node.js 24.21.0, выполняет npm ci и vite build, а содержимое dist упаковывается в client.war. Сборка с base: "./" позволяет открыть клиент по адресу /client/ на Tomcat.

Адрес сервиса коллекции подставляется при сборке из свойства Gradle vehiclesApi (./gradlew :client:war -PvehiclesApi=https://<хост>:<порт>/api/v1). Если свойство не задано, используется тот же хост с портом 8443. Автомастерская вызывается по относительному адресу /api/v1/shop.

6.2. Возможности

Таблица 10 — Вкладки клиента и используемые операции API

Вкладка	Возможности	Операции
Коллекция	Сортировка по нескольким полям с выбором направления; фильтры «поле — оператор — значение»; размер и номер страницы, переход назад и вперёд; выбор строки, карточка элемента, замена целиком, изменение отмеченных полей, удаление с подтверждением, операции автомастерской для выбранного элемента	GET, PUT, PATCH, DELETE /vehicles; обе операции shop
Добавить	Форма создания; показ созданного элемента и адреса из Location	POST /vehicles
Поиск и статистика	Поиск по ID, элемент с максимальной мощностью, количество с пробегом больше заданного, элементы с типом меньше заданного	GET /vehicles/{id} и три дополнительные операции
Автомастерская	Скрутить пробег и добавить колёса по ID; показ состояния элемента после перенаправления	обе операции shop

6.3. Человекочитаемое представление

Списки выводятся таблицей с русскими заголовками столбцов. Даты и числа форматируются по локали ru-RU, тип показывается по-русски («Лодка», «Самолёт»). Отдельный элемент описывается абзацем, например: «„Aurora“ — лодка с идентификатором 1, добавлен в коллекцию …; мощность двигателя — 150, вместимость — 4,5, пробег — 1 500,5, колёс — 4». Количество выводится фразой с согласованием числительного. Ответ 204 сообщает, что ничего не найдено. Над таблицей показано, сколько найдено элементов и какая страница из скольких открыта.

6.4. Сообщения об ошибках

Ответ с кодом ошибки клиент превращает в объект ApiError (сервис, код, сообщение) и показывает плашку с заголовком вида «Сервис коллекции: 422 — недопустимые данные» и текстом из <error> сервиса, например «enginePower: значение должно быть больше 0». Клиент сам данные не проверяет и отправляет их как есть, поэтому пользователь видит

именно ответ сервиса на невалидные данные. Если сервис недоступен или браузер не доверяет его сертификату, выводится сообщение «Не удалось связаться с сервисом. Проверьте, что он запущен и что браузер доверяет его сертификату».

7. HTTPS и самоподписанные сертификаты

7.1. Сертификаты

Для каждого сервера выпущен свой самоподписанный сертификат: издатель совпадает с субъектом C=RU, O=ITMO, OU=SOA, CN=localhost. Ключ RSA 2048 бит, подпись SHA256withRSA, срок действия — с 22.10.2026 по 22.10.2027. В расширение Subject Alternative Name включены IP 127.0.0.1, DNS localhost и DNS helios.cs.ifmo.ru, поэтому один сертификат подходит и для локального запуска, и для helios.

Таблица 11 — Файлы ключей и сертификатов

Файл	Назначение	Где лежит
vehicles.p12	Ключ и сертификат WildFly (PKCS12)	standalone/configuration
vehicles.crt	Сертификат WildFly для проверки доверия в shop-service и в браузере	на helios: ~/soa/lab2/certs/vehicles.crt
shop.p12	Ключ и сертификат Tomcat (PKCS12)	conf
shop.crt	Сертификат Tomcat для добавления в доверенные браузера	certs

7.2. WildFly

В подсистеме Elytron создана цепочка vehiclesKS → vehiclesKM → vehiclesSSC (хранилище ключей, менеджер ключей, серверный SSL-контекст с TLSv1.3 и TLSv1.2). Пароль хранилища берётся из переменной окружения KEYSTORE_PASSWORD. HTTPS-слушатель Undertow переключён на vehiclesSSC, HTTP-слушатель удалён, и порт 8080 сервер больше не открывает. Коннектор remoting перенесён с HTTP-слушателя на HTTPS.

```
<key-store name="vehiclesKS">
    <credential-reference clear-text="${env.KEYSTORE_PASSWORD}"/>
    <implementation type="PKCS12"/>
    <file path="vehicles.p12" relative-to="jboss.server.config.dir"/>
</key-store>
<key-manager name="vehiclesKM" key-store="vehiclesKS">
    <credential-reference clear-text="${env.KEYSTORE_PASSWORD}"/>
</key-manager>
<server-ssl-context name="vehiclesSSC" protocols="TLSv1.3 TLSv1.2" key-manager="vehiclesKM"/>

<!-- http-listener "default" удалён -->
<https-listener name="https" socket-binding="https"
                ssl-context="vehiclesSSC" enable-http2="true"/>
```

7.3. Tomcat

В conf/server.xml удалён HTTP-коннектор на 8080. Остался один коннектор, и он принимает только TLS. Порт, адрес привязки, порт выключения и пароль хранилища задаются системными свойствами из bin/setenv.sh.

```
<Server port="${shop.shutdown.port}" shutdown="SHUTDOWN">
  ...
  <Connector port="${shop.https.port}" address="${shop.bind.address}"
             protocol="org.apache.coyote.http11.Http11NioProtocol"
             maxThreads="50" SSLEnabled="true">
    <UpgradeProtocol className="org.apache.coyote.http2.Http2Protocol"/>
    <SSLHostConfig protocols="TLSv1.2+TLSv1.3">
      <Certificate certificateKeystoreFile="conf/shop.p12"
                   certificateKeystoreType="PKCS12"
                   certificateKeystorePassword="${shop.keystore.password}"/>
    </SSLHostConfig>
  </Connector>
```

7.4. Доверие между участниками

shop-service доверяет только сертификату vehicles.crt из SSL-бандла (раздел 5.2), а системное хранилище доверенных сертификатов JVM для этого соединения не используется. Имя хоста проверяет HttpClient по Subject Alternative Name. Браузер должен доверять обоим сертификатам: их добавляют в доверенные или один раз открывают адреса сервисов и принимают исключение. Иначе клиент покажет сообщение из раздела 6.4.

8. Конфигурация серверов и развёртывание на helios

8.1. WildFly 41.0.1.Final

Таблица 12 — Изменения в WildFly

Файл	Что сделано
bin/standalone.conf	Экспорт DB_URL, DB_USER, DB_PASSWORD, KEYSTORE_PASSWORD; на helios также _JAVA_OPTIONS и port-offset = 4789
modules/org/postgresql/main/module.xml	Модуль драйвера PostgreSQL; зависимости java.se, jakarta.transaction.api
modules/org/postgresql/main/postgresql-42.7.14.jar	JAR драйвера
standalone/configuration/standalone.xml	Драйвер postgresql и источник VehiclesDS на ${env.*}; Elytron vehiclesKS → vehiclesKM → vehiclesSSC; https-listener на vehiclesSSC; http-listener удалён; remoting на https
standalone/configuration/standalone.xml.orig	Резервная копия исходного файла
standalone/configuration/vehicles.p12	Ключ и сертификат
standalone/deployments/vehicles.war	Развёртываемое приложение первого сервиса

Параметры подключения к базе данных и пароль хранилища ключей в standalone.xml не записаны: они подставляются выражениями ${env.DB_URL}, ${env.DB_USER}, ${env.DB_PASSWORD} и ${env.KEYSTORE_PASSWORD}. Пул соединений VehiclesDS — от 1 до 10, соединение проверяется запросом SELECT 1 при выдаче из пула.

8.2. Tomcat 11.0.27

Таблица 13 — Изменения в Tomcat

Файл	Что сделано
bin/setenv.sh	CATALINA_OPTS со свойствами shop.* (порт, адрес, порт выключения, пароль), VEHICLES_BASE_URL, VEHICLES_CERT; локально JAVA_HOME, на helios _JAVA_OPTIONS
conf/server.xml	Порт выключения ${shop.shutdown.port}; HTTP-коннектор 8080 удалён; единственный HTTPS-коннектор на ${shop.https.port} / ${shop.bind.address} с conf/shop.p12
conf/shop.p12	Ключ и сертификат

Файл	Что сделано
webapps/ROOT.war	Второй сервис (контекст /)
webapps/client.war	Клиентское приложение (контекст /client)
webapps/* (прочее)	Удалены стандартные приложения Tomcat (docs, examples, manager и др.)

Вне каталогов серверов на helios лежит ~/soa/lab2/certs/vehicles.crt. Это сертификат, которому доверяет второй сервис, и путь к нему указан в VEHICLES_CERT.

8.3. Сборка и запуск

```
./gradlew build
# vehicles-service/build/libs/vehicles.war -> wildfly/standalone/deployments/
# shop-service/build/libs/ROOT.war         -> tomcat/webapps/
# client/build/libs/client.war             -> tomcat/webapps/
# адрес сервиса коллекции для клиента:
#   ./gradlew :client:war -PvehiclesApi=https://<хост>:<порт>/api/v1

wildfly-41.0.1.Final/bin/standalone.sh
apache-tomcat-11.0.27/bin/startup.sh
```

На helios WildFly запускается со смещением портов 4789, поэтому HTTPS-порт первого сервиса — 8443 + 4789 = 13232. Порты Tomcat заданы свойствами shop.https.port и shop.shutdown.port в setenv.sh, так что один и тот же server.xml работает на обеих машинах.

9. Проверка

Команда ./gradlew build выполнена 29.10.2026 и завершилась успешно: собраны vehicles.war, ROOT.war и client.war. Модульный тест XmlMappingTest сервиса коллекции прошёл (7 тестов, 0 ошибок). Тесты проверяют XML-представление Vehicle, VehiclePage, VehicleList, CountResult и ErrorResponse, подстановку значений по умолчанию в VehicleInput и то, что непереданные поля VehiclePatch остаются пустыми (null).

В журнале WildFly (wildfly.log) зафиксированы развёртывание приложения и запуск сервера: «WFLYSRV0010: Deployed "vehicles.war"» и «WildFly 41.0.1.Final … started in 7221ms».

Для демонстрации обработки ошибок подготовлены сценарии из таблицы 14. Ожидаемые ответы определены по коду обработчиков. В запросах POST тело во всём остальном корректно, нарушено только указанное поле.

Таблица 14 — Сценарии и ожидаемые ответы

Запрос	Ожидаемый ответ
POST /vehicles без элемента name	422 «name: поле обязательно»
POST /vehicles с <enginePower>abc</enginePower>	400 «Тело запроса не является корректным XML»
POST /vehicles с coordinates.y = −1000	422 «coordinates.y: значение должно быть больше -943»
GET /vehicles/abc	400 «id: ожидается целое число»
GET /vehicles/{несуществующий id}	404 «Транспортное средство не найдено»
GET /vehicles?size=0	422 «size: значение должно быть от 1 до 100»
GET /vehicles?sort=color	400 «sort: неизвестное поле сортировки 'color'»
GET /vehicles?filter=type[gt]=BOAT	422 «filter: оператор 'gt' неприменим к полю 'type'»
GET /vehicles/type/less-than/CAR	400 «type: неизвестное значение, допустимые: PLANE, …»
PATCH /api/v1/shop/add-wheels/1/0	422 «number-of-wheels: значение должно быть больше 0»

Запрос	Ожидаемый ответ
PATCH /api/v1/shop/fix-distance/1 при остановленном WildFly	503 «Сервис коллекции недоступен»
Запрос по http:// на порт 8080 любого сервера	Соединение не устанавливается: HTTP-слушателей нет

10. Выводы

По спецификации лабораторной работы №1 реализованы два веб-сервиса и клиентское приложение. Сервис коллекции написан на JAX-RS и развёрнут в WildFly 41: CRUD-операции, три дополнительные операции, сортировка и фильтрация по любой комбинации полей, постраничный вывод, данные в XML, ошибки в едином формате с кодами 400, 404, 405, 406, 415 и 422. Автомастерская написана на Spring MVC REST, развёрнута в Tomcat 11 и выполняет свои операции через REST API первого сервиса, возвращая 303 See Other на изменённый ресурс.

Оба сервера принимают только HTTPS с самоподписанными сертификатами, HTTP-коннекторы удалены. Межсервисный вызов проверяет сертификат первого сервиса по закреплённому файлу, а не по системному хранилищу. Пароли и параметры БД вынесены в переменные окружения, поэтому конфигурация серверов переносится на helios без правки XML.

Клиент на Vue даёт доступ ко всем операциям обоих сервисов, показывает данные таблицей и текстом и выводит сообщения сервисов об ошибках, в том числе о невалидных данных.

11. Источники

1. Jakarta RESTful Web Services 4.0. https://jakarta.ee/specifications/restful-ws/4.0/ (дата обращения: 29.10.2026).

2. Jakarta Persistence 3.2. https://jakarta.ee/specifications/persistence/3.2/ (дата обращения: 29.10.2026).

3. WildFly Documentation. https://docs.wildfly.org/ (дата обращения: 29.10.2026).

4. Apache Tomcat 11. SSL/TLS Configuration How-To. https://tomcat.apache.org/tomcat-11.0-doc/ssl-howto.html (дата обращения: 29.10.2026).

5. Spring Boot Reference. SSL. https://docs.spring.io/spring-boot/reference/features/ssl.html (дата обращения: 29.10.2026).

6. Spring Framework Reference. REST Clients. https://docs.spring.io/spring-framework/reference/integration/rest-clients.html (дата обращения: 29.10.2026).

7. RFC 9110. HTTP Semantics (303 See Other). https://www.rfc-editor.org/rfc/rfc9110 (дата обращения: 29.10.2026).

8. Vue.js Guide. https://vuejs.org/guide/ (дата обращения: 29.10.2026).

9. Исходный код лабораторной работы. https://github.com/xxxxxxhs/soa_lab2 (дата обращения: 29.10.2026).

Приложение А. Выполнение пунктов задания

Таблица 15 — Соответствие требованиям задания

Требование	Как выполнено	Раздел
Первый сервис на JAX-RS в WildFly	vehicles-service, vehicles.war в WildFly 41.0.1.Final	4, 8.1
Второй сервис на Spring MVC REST в Tomcat, вызывает API первого	shop-service, ROOT.war в Tomcat 11.0.27; VehiclesClient (RestClient)	5, 8.2
Все функции API в соответствии со спецификацией	CRUD, три дополнительные операции, две операции автомастерской; коды ответов по таблицам 5 и 8	4.2, 5.1
HTTPS с самоподписанным сертификатом	Elytron vehiclesSSC в WildFly, SSL-коннектор в Tomcat	7
HTTP без шифрования запрещён	HTTP-слушатель WildFly и HTTP-коннектор Tomcat удалены	7.2, 7.3
Клиент на веб-фреймворке, работающий на helios	Vue 3 + Vite, статическая сборка в client.war на Tomcat	6.1
Полный набор возможностей API, сортировка, фильтрация, страницы	Вкладки «Коллекция», «Добавить», «Поиск и статистика», «Автомастерская»	6.2
Человекочитаемый вид	Таблицы, абзацы-описания, локализованные даты, числа и типы	6.3
Информирование об ошибках, в том числе о невалидных данных	Плашка с кодом и сообщением сервиса	6.4
Развёртывание на helios	Те же WAR и конфигурация; port-offset 4789, переменные окружения	8

Приложение Б. Структура репозитория

```
soa_lab2/
├── build.gradle, settings.gradle, gradlew   общая сборка, Java 17
├── vehicles-service/                        первый сервис (JAX-RS)
│   ├── src/main/java/ru/itmo/soa/vehicles/
│   │   ├── VehiclesApplication.java         @ApplicationPath("/api/v1")
│   │   └── api/ service/ model/ dto/ query/ validation/ xml/ error/
│   ├── src/main/resources/META-INF/persistence.xml
│   ├── src/main/webapp/WEB-INF/jboss-web.xml
│   └── src/test/.../XmlMappingTest.java
├── shop-service/                            второй сервис (Spring MVC REST)
│   ├── src/main/java/ru/itmo/soa/shop/
│   │   ├── ShopApplication.java             SpringBootServletInitializer
│   │   └── api/ service/ client/ dto/ xml/ error/
│   └── src/main/resources/application.properties
└── client/                                  клиент (Vue 3 + Vite)
    ├── build.gradle                         node-gradle, сборка client.war
    └── src/
        ├── App.vue, api.js, labels.js, notice.js
        └── components/                      CollectionView, CreateView,
                                             SearchView, ShopView и др.
```